

PROJECT: Multi-Region Disaster Recovery Plan

Services: EC2, RDS, Route 53, CloudFormation

Description: Set up a disaster recovery architecture where data and applications are replicated across two regions. Automate failover using Route 53 health checks.

Skills: High availability, failover planning, cross-region replication.

Definition: This project implements a Multi-Region Disaster Recovery architecture on AWS to ensure application availability during failures. The application is deployed in a primary region (Mumbai) and replicated to a secondary region (Virginia). AWS Route 53 health checks and failover routing automatically redirect traffic to the backup server when the primary server becomes unavailable. This architecture improves high availability, reliability, and business continuity.

Scope of the Project: The scope of this project includes deploying application servers using Amazon EC2 and databases using Amazon RDS in a primary region.

- A cross-region read replica is configured to replicate database data to the backup region.
- Route 53 DNS failover and health checks monitor the primary server and automatically switch traffic to the backup server during failure.
- The project demonstrates disaster recovery, automated failover, and multi-region cloud architecture on AWS.

Methodologies:

1. Active-Passive Disaster Recovery with DNS Failover: Method used in this Project

- In this architecture, the primary application runs in Mumbai (active region) and the backup environment in Virginia remains on standby (passive region).
 - AWS Route 53 health checks monitor the primary server, and if it becomes unhealthy, DNS failover automatically redirects traffic to the backup region.
 - This method is suitable because it ensures high availability while keeping infrastructure cost low. Only the primary region handles traffic, while the secondary region acts as a backup and activates during failure.
- It also provides automatic failover using Route 53, ensuring minimal downtime during outages.

2. Backup and Restore: Alternative Method

- Data backups are stored in another region and restored when a disaster occurs.
- Recovery time is slow because infrastructure and data must be rebuilt during failure.

3. Active-Active (Multi-Region)

- Both regions are active and serve traffic simultaneously.
- It is more expensive and complex, requiring load balancing and full infrastructure duplication.

Services Used in this Project:

1. Amazon EC2 (Elastic Cloud Computing)

- **Definition:** Amazon EC2 is a cloud service that provides scalable virtual servers to run applications on AWS.
- **Purpose:** EC2 is used to host the web application servers in both the primary (Mumbai) and backup (Virginia) regions.
- **Role in this Project:** EC2 runs the application and serves user requests, while Route 53 redirects traffic to the backup EC2 if the primary server fails.

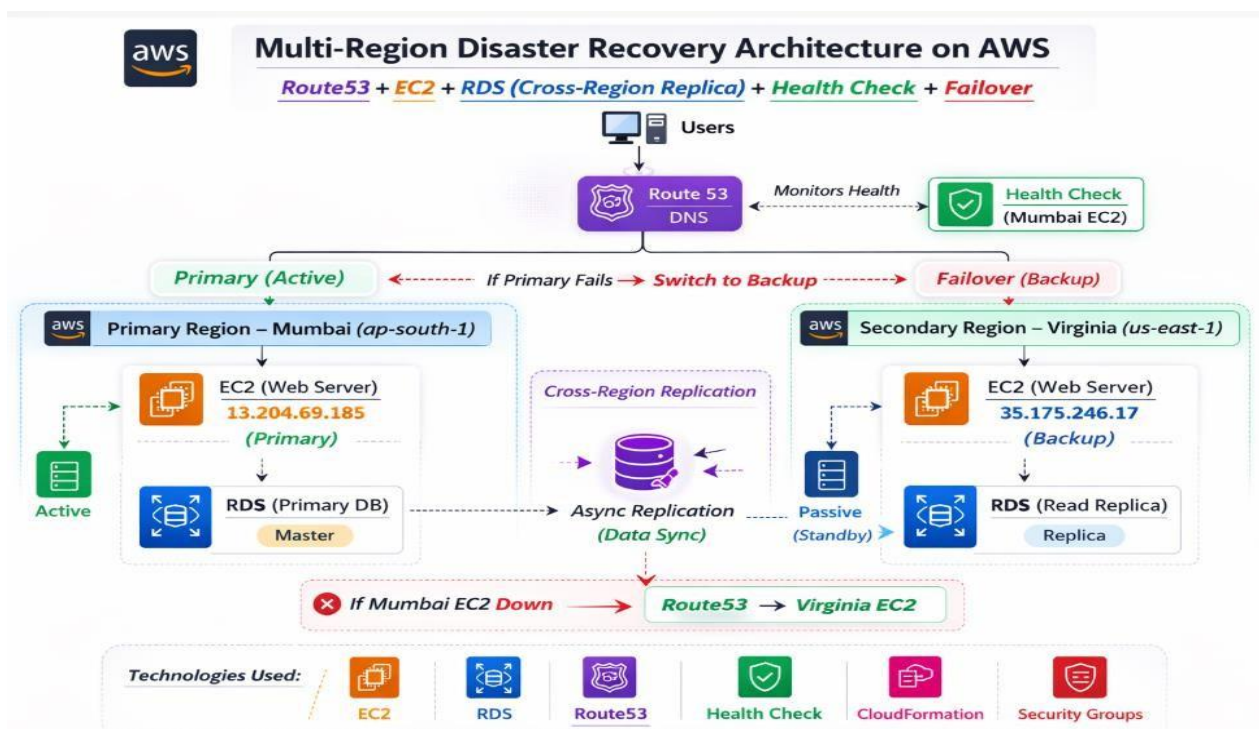
2. RDS (Relational Database Service)

- **Definition:** Amazon RDS is a managed AWS service used to create, operate, and scale relational databases in the cloud.
- **Purpose:** RDS stores the application data and replicates it from the primary database in Mumbai to a read replica in Virginia.
- **Role in this Project:** RDS ensures database availability by maintaining a primary database and a cross-region replica for disaster recovery.

3. Route 53

- **Definition:** Amazon Route 53 is a scalable DNS and domain name service used to route user traffic to AWS resources.
- **Purpose:** Route 53 monitors the primary server using health checks and manages DNS failover between regions.
- **Role in this Project:** Route 53 automatically redirects user traffic from the Mumbai server to the Virginia backup server if the primary server becomes unavailable.

Architecture Diagram:



Implementation and Execution of a Multi-Region Disaster Recovery Plan

STEP 1: Launch EC2 Instance in Primary Region (Mumbai)

- Open AWS Management Console.
- Go to EC2 → Launch Instance.
- Select Region: Asia Pacific (Mumbai).
- Choose Amazon Linux AMI.
- Select Instance type: t3. micro.
- Configure Security Group:
 - SSH → Port 22
 - HTTP → Port 80
- Launch the instance.

STEP 2: Install Web Application on Mumbai EC2

```
aws [Alt+S] Asia Pacific (Mumbai) Pavan (2074-5447-8223) Pavan  
~ #  
~ ##### Amazon Linux 2023  
~ \#####\br/>~ \###! \##/  
~ V-> https://aws.amazon.com/linux/amazon-linux-2023  
~ ~~~~~  
~ ~~~~~  
~ ~~~~~  
~ ~~~~~  
[ec2-user@ip-10-0-2-169 ~]$ sudo yum update -y  
Amazon Linux 2023 Kernel Livepatch repository 253 kB/s | 30 kB 00:00  
Last metadata expiration check: 0:00:01 ago on Sat Mar 7 08:15:46 2026.  
Dependencies resolved.  
Nothing to do.  
Complete!  
[ec2-user@ip-10-0-2-169 ~]$
```

```

aws  [P] [X] [Q Search] [Alt+S] [X] [P] [Q] [G] Asia Pacific (Mumbai) Pavan
[ec2-user@ip-10-0-2-169 ~]$ sudo yum install httpd -y
Last metadata expiration check: 0:01:28 ago on Sat Mar 7 08:15:46 2026.
Dependencies resolved.
=====
Package                        Architecture      Version           Repository        Size
=====
Installing:
httpd                          x86_64            2.4.66-1.amzn2023.0.1  amazonlinux      47 k
Installing dependencies:
apr                            x86_64            1.7.5-1.amzn2023.0.4   amazonlinux      129 k
apr-util                       x86_64            1.6.3-1.amzn2023.0.2   amazonlinux      97 k
apr-util-ldap                  x86_64            1.6.3-1.amzn2023.0.2   amazonlinux      13 k
generic-logos-httpd           noarch            18.0.0-12.amzn2023.0.3  amazonlinux      19 k
httpd-core                     x86_64            2.4.66-1.amzn2023.0.1  amazonlinux      1.4 M
httpdfilesystem                noarch            2.4.66-1.amzn2023.0.1  amazonlinux      13 k
httpd-tools                    x86_64            2.4.66-1.amzn2023.0.1  amazonlinux      81 k
libbrotli                      x86_64            1.0.9-4.amzn2023.0.2   amazonlinux      315 k
mailcap                         noarch            2.1.49-3.amzn2023.0.3  amazonlinux      33 k
Installing weak dependencies:
apr-util-openssl               x86_64            1.6.3-1.amzn2023.0.2   amazonlinux      15 k
mod_http2                      x86_64            2.0.27-1.amzn2023.0.3  amazonlinux      166 k
mod_lua                         x86_64            2.4.66-1.amzn2023.0.1  amazonlinux      60 k
Transaction Summary

```

STEP 3: Installing and Configuring Apache Web Server on EC2 Instance

```
[ec2-user@ip-10-0-2-169 ~]$ sudo systemctl start httpd
[ec2-user@ip-10-0-2-169 ~]$ sudo systemctl enable httpd
Created symlink /etc/systemd/system/multi-user.target.wants/httpd.service → /usr/lib/systemd/system/httpd.service.
[ec2-user@ip-10-0-2-169 ~]$
```

STEP 4: Create a web page

```
[ec2-user@ip-10-0-2-169 ~]$ echo "Primary Server - Mumbai Region" | sudo tee /var/www/html/index.html
Primary Server - Mumbai Region
[ec2-user@ip-10-0-2-169 ~]$
```

STEP 5: Now open the browser:



STEP 6: Create RDS Database in Mumbai Region

- Go to AWS Console → RDS.
- Click Create Database

A screenshot of the AWS Management Console 'Create database' page. The 'Settings' section shows 'DB instance identifier' as 'mumbai-primary-db'. The 'Credentials Settings' section shows 'Master username' as 'admin'. Under 'Credentials management', 'Self managed' is selected. The 'Master password' field is filled with a strong password, indicated by a 'Very strong' strength meter. The 'Confirm master password' field is also filled.A screenshot of the AWS Management Console 'Databases' page. A green banner at the top states 'Successfully created database mumbai-primary-db'. Below, a table lists the database instance.

DB identifier	Status	Role	Engine	Upgrade rollout order	Region...	Size	Recommendation
mumbai-primary-db	Available	Instance	MySQL Co...	SECOND	ap-south-1b	db.t3.micro	

STEP 7: Create DB Subnet Group in Virginia

- Switch region to US East (N. Virginia).
- Go to RDS → Subnet Groups.
- Click Create DB Subnet Group.
- Select:
 - VPC
 - Two private subnets
- Create subnet group.

The screenshot shows the 'Create DB subnet group' page in the AWS Management Console. The breadcrumb navigation is 'Aurora and RDS > Subnet groups > Create DB subnet group'. The page title is 'Create DB subnet group' with a subtitle: 'To create a new subnet group, give it a name and a description, and choose an existing VPC. You will then be able to add subnets related to that VPC.'

Subnet group details

Name
You won't be able to modify the name after your subnet group has been created.
Input field: `virginia-db-subnet-group`
Must contain from 1 to 255 characters. Alphanumeric characters, spaces, hyphens, underscores, and periods are allowed.

Description
Input field: `DR subnet group`

VPC
Choose a VPC identifier that corresponds to the subnets you want to use for your DB subnet group. You won't be able to choose a different VPC identifier after your subnet group has been created.
Dropdown menu: `project-vpc (vpc-06b7f4cf433fe73f5)`
4 Subnets, 2 Availability Zones

The screenshot shows the 'Subnet groups' page in the AWS Management Console. A green success banner at the top reads: 'Successfully created virginia-db-subnet-group. View subnet group'. The breadcrumb navigation is 'Aurora and RDS > Subnet groups'. The left sidebar shows 'Aurora and RDS' with a list of links: Dashboard, Databases, Performance insights, Snapshots, Exports in Amazon S3, Automated backups, and Reserved instances.

Subnet groups (1)

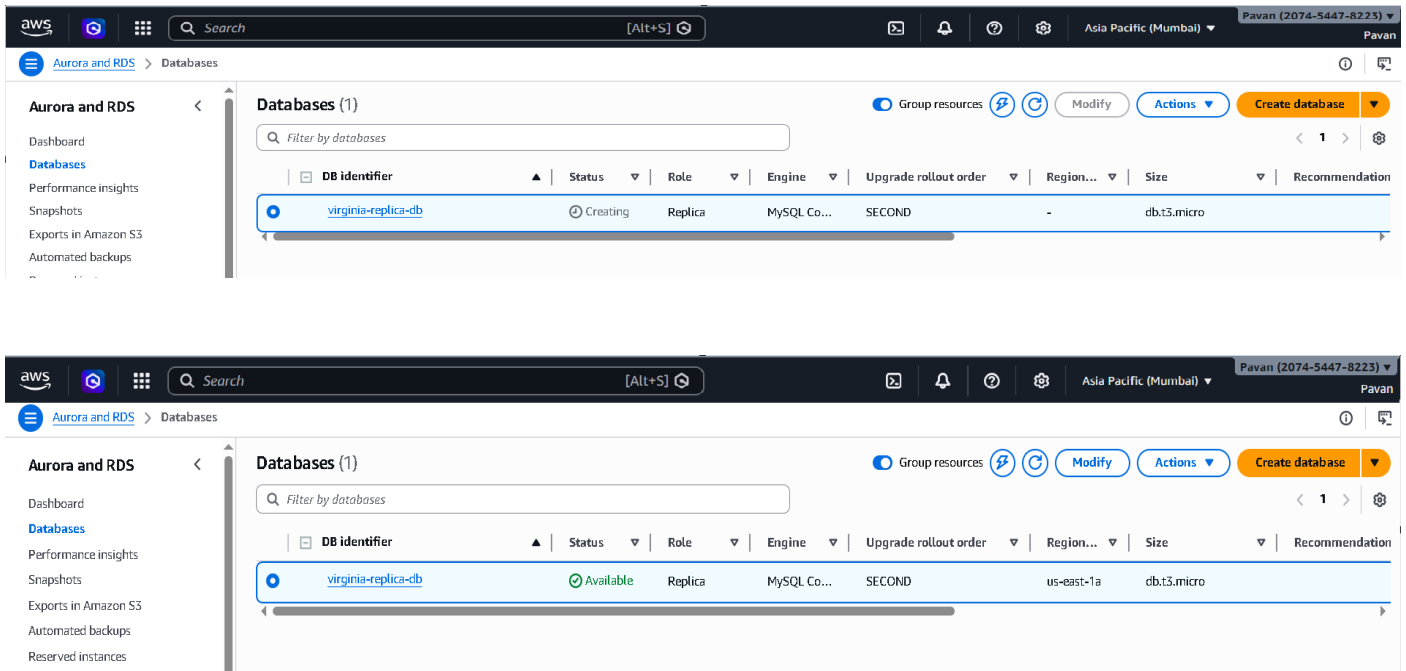
Filter by subnet group

☒	Name	Description	Status	VPC
☒	virginia-db-subnet-group	DR subnet group	Complete	vpc-06b7f4cf433fe73f5

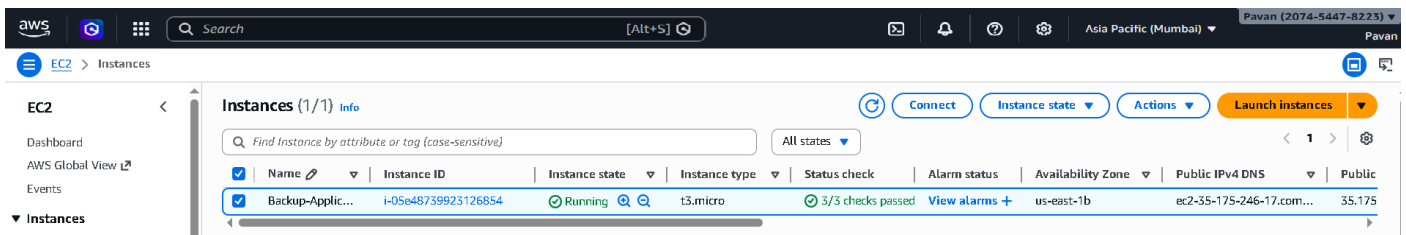
Buttons: Refresh, Edit, Delete, Create DB subnet group

STEP 8: Create Cross-Region RDS Read Replica

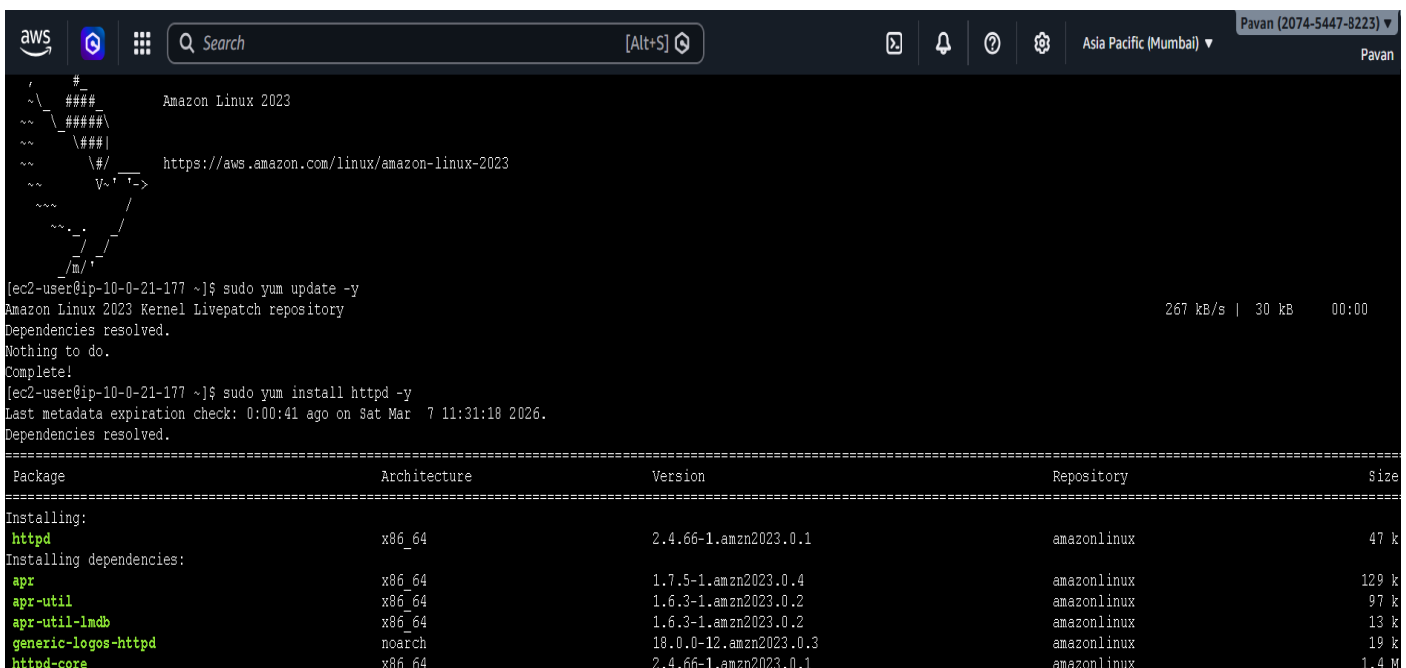
- Go to RDS → Databases.
- Select the Mumbai Primary DB.
- Click:
 - Actions → Create cross-region replica
- Change region to:
 - US East (N. Virginia)
- Select the Virginia subnet group.
- Create replica.



STEP 9: Launch Backup EC2 in Virginia Region



STEP 10: Install web server



STEP 11: Create backup page

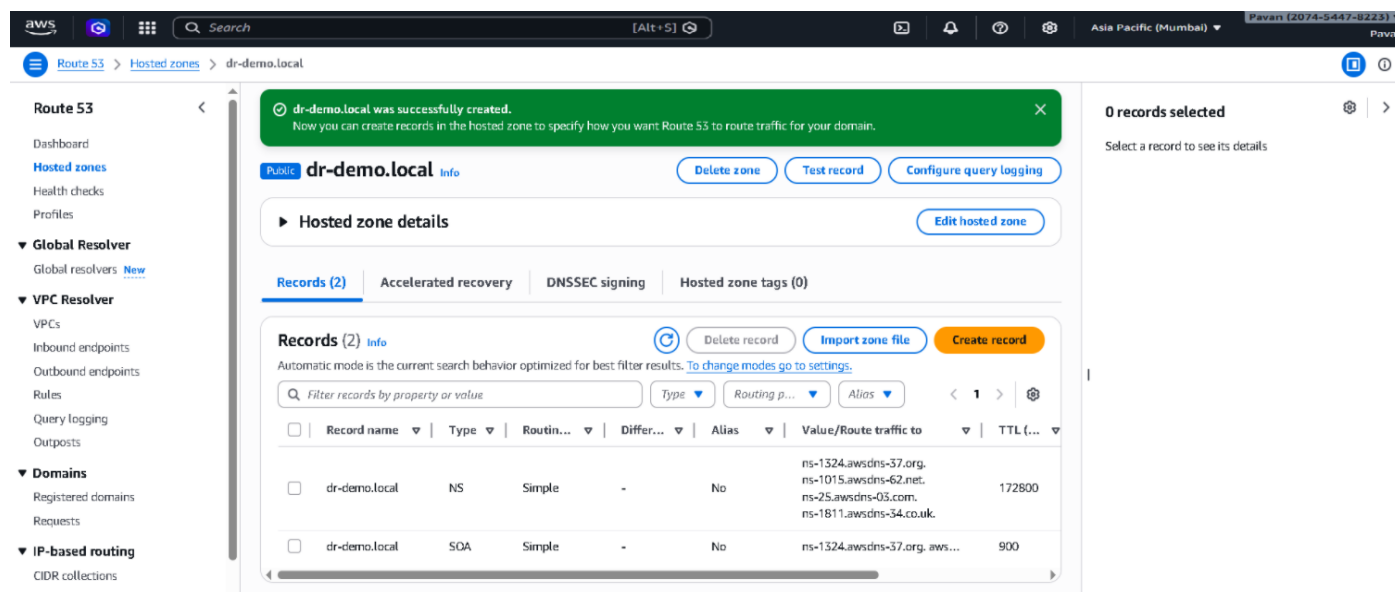
```
ec2-user@ip-10-0-21-177 ~]$ echo "Backup Server - Virginia Region" | sudo tee /var/www/html/index.html
Backup Server - Virginia Region
ec2-user@ip-10-0-21-177 ~]$
```

STEP 12: Test in Browser



STEP 13: Create Route 53 Hosted Zone

- Go to Route 53 → Hosted Zones.
- Click Create Hosted Zone.



STEP 14: Create Route 53 Health Check

- Go to Route 53 → Health Checks.
- Click Create Health Check.

aws

Search

[Alt+S]

Asia Pacific (Mumbai)

Pavan (2074-5447-8223)

Pavan

Route 53

Health checks

mumbai-ec2-health

mumbai-ec2-health

Info

Delete

Invert

Disable

Edit

Configuration

ID

b07a036d-54be-455e-9f02-d6fe21fa9f41

URL

http://13.204.69.185:80/

Specified endpoint by

IP address

State

Enabled

Status

Healthy

Inverted

No

STEP 15: Create Route 53 Failover Records

- Go to Hosted Zone → dr-demo.local.

aws

Search

[Alt+S]

Asia Pacific (Mumbai)

Pavan (2074-5447-8223)

Pavan

Route 53

Hosted zones

dr-demo.local

Record for dr-demo.local was successfully created.

Route 53 propagates your changes to all of the Route 53 authoritative DNS servers within 60 seconds. Use "View status" button to check propagation status.

View status

Public dr-demo.local

Info

Delete zone

Test record

Configure query logging

Hosted zone details

Edit hosted zone

Records (3)

Accelerated recovery

DNSSEC signing

Hosted zone tags (0)

Records (1/3)

Info

Delete record

Import zone file

Create record

Automatic mode is the current search behavior optimized for best filter results. To change modes go to settings.

Filter records by property or value

Type

Routing p...

Alias

< 1 >

Record name

Type

Routing...

Differ...

Alias

Value/Route traffic to

TTL (...)

☐

dr-demo.local

NS

Simple

-

No

ns-1324.awsdns-37.org.
ns-1015.awsdns-62.net.
ns-25.awsdns-03.com.
ns-1811.awsdns-34.co.uk.

172800

☐

dr-demo.local

SOA

Simple

-

No

ns-1324.awsdns-37.org. aws...

900

☒

app.dr-demo.l...

A

Failover

Primary

No

13.204.69.185

60

Record details

Edit record

Record name

app.dr-demo.local

Record type

A

Value

13.204.69.185

Alias

No

TTL (seconds)

60

Routing policy

Failover

Failover record type

Primary

Health check ID

b07a036d-54be-455e-9f02-d6fe21fa9f41

Record Id

Mumbai-Primary

STEP 16: Secondary

aws

Search

[Alt+S]

Asia Pacific (Mumbai)

Pavan (2074-5447-8223)

Pavan

Route 53

Hosted zones

dr-demo.local

Record for dr-demo.local was successfully created.

Route 53 propagates your changes to all of the Route 53 authoritative DNS servers within 60 seconds. Use "View status" button to check propagation status.

View status

Public dr-demo.local

Info

Delete zone

Test record

Configure query logging

Hosted zone details

Edit hosted zone

Records (4)

Accelerated recovery

DNSSEC signing

Hosted zone tags (0)

Records (1/4)

Info

Delete record

Import zone file

Create record

Automatic mode is the current search behavior optimized for best filter results. To change modes go to settings.

Filter records by property or value

Type

Routing p...

Alias

< 1 >

Record name

Type

Routing...

Differ...

Alias

Value/Route traffic to

TTL (...)

☐

dr-demo.local

NS

Simple

-

No

ns-1324.awsdns-37.org.
ns-1015.awsdns-62.net.
ns-25.awsdns-03.com.
ns-1811.awsdns-34.co.uk.

172800

☐

dr-demo.local

SOA

Simple

-

No

ns-1324.awsdns-37.org. aws...

900

☐

app.dr-demo.l...

A

Failover

Primary

No

13.204.69.185

60

☒

app.dr-demo.l...

A

Failover

Secondary

No

35.175.246.17

60

Record details

Edit record

Record name

app.dr-demo.local

Record type

A

Value

35.175.246.17

Alias

No

TTL (seconds)

60

Routing policy

Failover

Failover record type

Secondary

Record Id

Virginia-Backup

STEP 17: Test Disaster Recovery

- Simulate failure.
- Go to EC2 → Mumbai instance.
- Click:
- Stop Instance

Successfully initiated stopping of i-028162150f74b1027

Instances (1/2) Info

Find Instance by attribute or tag (case-sensitive) All states

Name	Instance ID	Instance state	Instance type	Status check	Alarm status	Availability Zone	Public IPv4 DNS	Public
Jenkins-CICD-...	i-0247679acf121bcae	Stopped	m7i-flex.large	-	View alarms +	ap-south-1b	-	-
primary-server-mumbai	i-028162150f74b1027	Stopped	t3.micro	-	View alarms +	ap-south-1b	-	-

i-028162150f74b1027 (primary-server-mumbai)

Details Status and alarms Monitoring Security Networking Storage Tags

▼ Instance summary Info

Instance ID i-028162150f74b1027	Public IPv4 address -	Private IPv4 addresses 10.0.2.169
IPv6 address -	Instance state Stopped	Public DNS -
Hostname type IP name: ip-10-0-2-169.ap-south-1.compute.internal	Private IP DNS name (IPv4 only) ip-10-0-2-169.ap-south-1.compute.internal	
Answer private resource DNS name	Instance type	Elastic IP addresses

-----> Now the primary server is down.

13.204.69.185

This site can't be reached

13.204.69.185 took too long to respond.

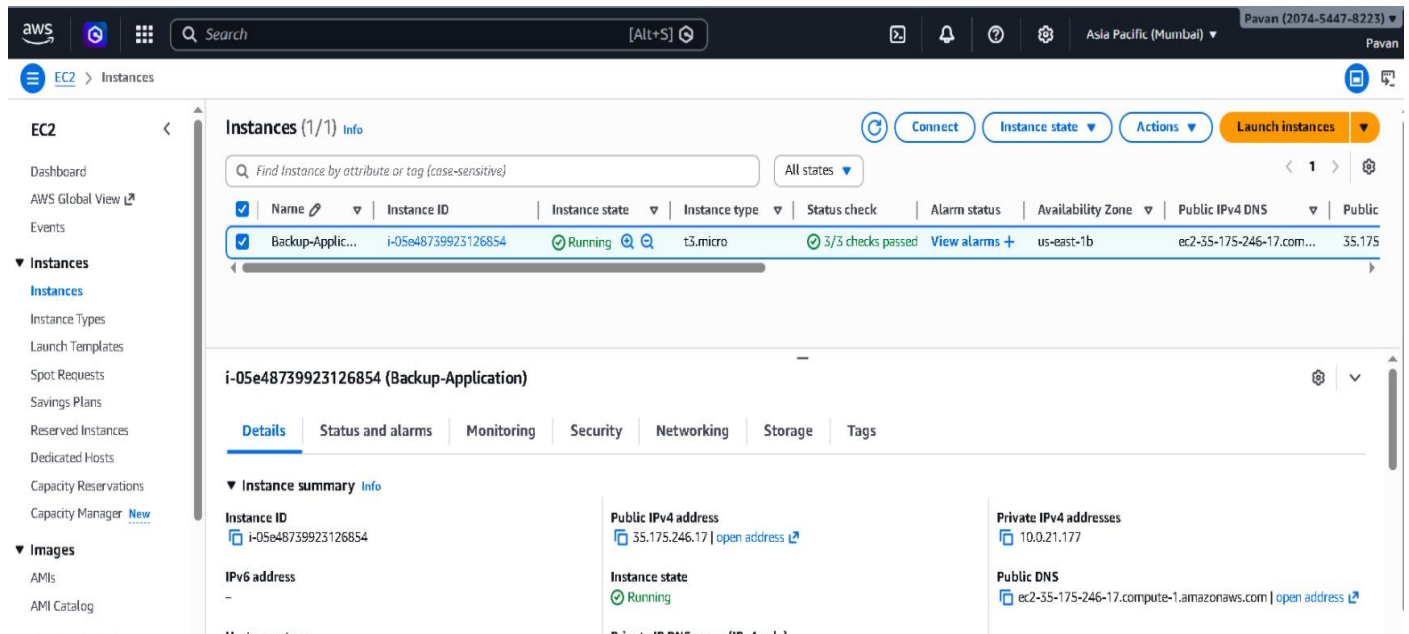
Try:

- Checking the connection
- Checking the proxy and the firewall
- Running Windows Network Diagnostics

ERR_CONNECTION_TIMED_OUT

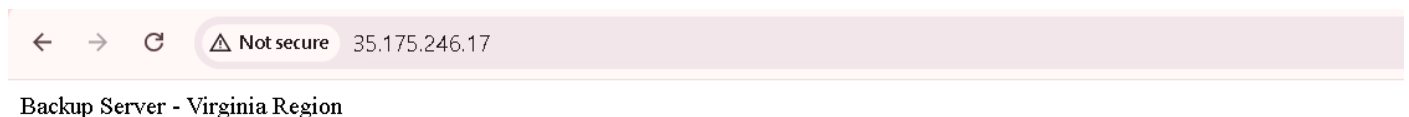
Reload Details

STEP 18: Now open



The screenshot shows the AWS Management Console for the EC2 Instances page. The instance 'Backup-Application' (ID: i-05e48739923126854) is in a 'Running' state. The console displays various tabs for the instance, including Details, Status and alarms, Monitoring, Security, Networking, Storage, and Tags. The instance summary shows the public IPv4 address 35.175.246.17 and the public DNS name ec2-35-175-246-17.compute-1.amazonaws.com.

STEP 19: This confirms failover works.



The screenshot shows a web browser with the address bar displaying '35.175.246.17' and the page title 'Backup Server - Virginia Region'.

Troubleshooting:

- **Issue:** RDS Subnet Group Not Appearing in Replica Creation

Problem: While creating the cross-region RDS replica, the subnet group was not visible in the dropdown.

Solution: Created a DB Subnet Group in the Virginia region with at least two subnets in different Availability Zones, which then allowed the replica creation.

- **Issue:** Route 53 Domain Not Resolving

Problem: The domain app.dr-demo.local did not open in the browser due to DNS resolution issues.

Solution: Verified Route 53 records and tested the application using EC2 public IP addresses, since .local domains are not publicly resolvable.

Conclusion:

This project successfully implemented a Multi-Region Disaster Recovery architecture on AWS to ensure application availability during failures. The system used Amazon EC2 for hosting the application, Amazon RDS for database management with cross-region replication, and Route 53 for health checks and automatic failover routing. When the primary server in the Mumbai region became unavailable, traffic was automatically redirected to the backup server in the Virginia region. This architecture improves high availability, reliability, and business continuity while demonstrating effective cloud disaster recovery practices.